Multi-Agent Manufacturing System

Course Name: Agentic AI

Institution Name: *Medicaps University – Datagami Skill Based Course*

Student Name(s) & Enrolment Number(s):

Sr no	Student Name	Enrolment Number
1.	Aaditya Shrivastava	EN22CS301007
2.	Aakansha Jaiswal	EN22CS301011
3.	Akanksha Sharma	EN22CS301080
4.	Akshita Bhawsar	EN22CS301098
5.	Abhishek Malviya	EN22CS301034

Group Name: Group 12D1

Project Number: AAI-*12*

Industry Mentor Name: Aashruti Shah

University Mentor Name: Prof. Nishant Shrivastava

Academic Year: 2025-2026

Problem Statement & Objectives

1. Problem Statement

The manufacturing industry requires efficient tools to transform conceptual ideas into structured technical descriptions and visual prototypes. However, this process is typically manual, time-consuming, and requires expertise in both design and data analysis, making it inefficient for rapid decision-making.

Existing AI solutions are often limited to single functionalities, such as text generation or image generation, and do not provide an integrated system capable of handling complete workflows. Additionally, they lack the ability to collaborate intelligently across multiple tasks, such as research, analysis, and report generation.

There is a need for an intelligent, integrated system that can:

- Generate structured manufacturing descriptions
- Create visual prototypes from textual input
- Retrieve relevant information from external sources
- Automate report generation
- Coordinate multiple tasks seamlessly using AI agents

To address this gap, this project proposes a Multi-Agent Multimodal Manufacturing System using Agentic AI, where specialized agents collaborate to perform tasks such as research and report generation, while integrating text and image generation capabilities. This enables a complete end-to-end automated pipeline, transforming simple user input into professional, multi-format outputs efficiently.

2. Project Objectives

- To develop a multimodal system that converts natural language input into structured text and visual prototypes
- To integrate LLM and image generation models for automated content creation
- To implement a multi-agent (Agentic AI) architecture for task coordination such as research and report generation
- To enable information retrieval and semantic search using a vector database
- To provide persistent storage and automated report generation
- To design a user-friendly system for efficient and fast output generation

3. Scope of the Project

The scope of this project includes the development of an AI-based multimodal and multi-agent system for manufacturing concept generation.

The project covers:

- Generation of structured manufacturing descriptions from natural language input
- Creation of visual prototypes using image generation models
- Implementation of a multi-agent system (Agentic AI) for research and report generation
- Integration of a vector database for storing and retrieving past concepts
- Development of an automated workflow pipeline from input to output
- Providing report generation and export functionality
- Designing a user-friendly interface for interaction

The project does NOT cover:

- Real-time manufacturing simulation
- CAD or 3D model generation
- Large-scale cloud deployment
- Multi-user authentication systems

4. Proposed Solution

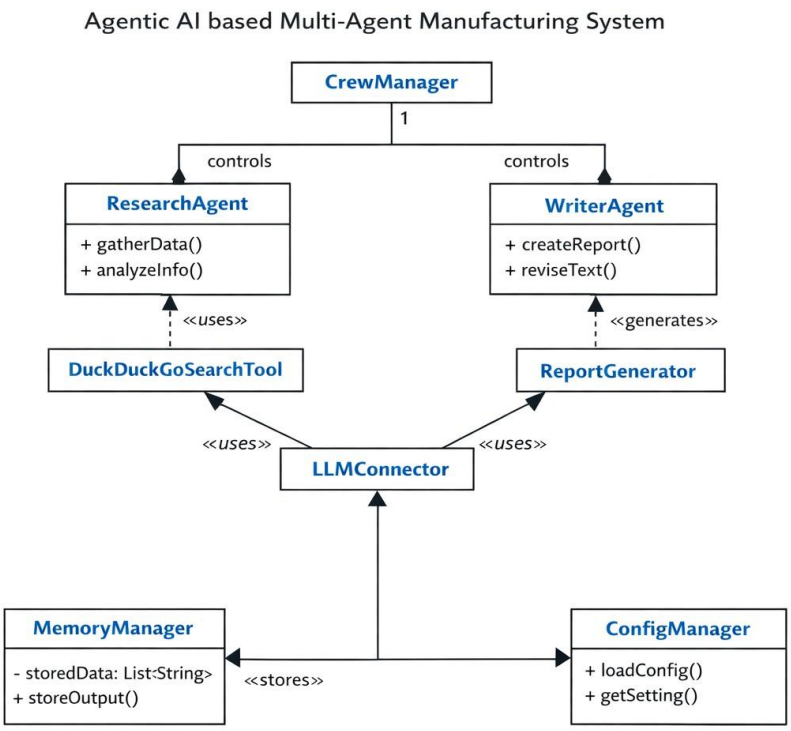
The proposed solution is to develop an AI-powered multimodal and multi-agent system that transforms a user's natural language manufacturing idea into a structured technical description and a visual prototype.

The system follows an integrated pipeline where:

- The user provides input through a simple interface
- A Large Language Model (LLM) generates a structured manufacturing description
- A prompt optimization step enhances input for better image generation
- An image generation model creates a realistic visual prototype
- A multi-agent system (Agentic AI) is used, where:
    - A Research Agent gathers relevant information
    - A Writer Agent refines and formats the final output
- A vector database (ChromaDB) stores and retrieves past concepts for semantic search
- The final output is delivered as text + image + downloadable report

5. LLD Components and Workflows

5.1 Class Design (Classes & UML Diagram)



UML Class Diagram Description – Agentic AI Multi-Agent Manufacturing System

This diagram represents a multi-agent system architecture where different AI agents collaborate to perform manufacturing-related tasks efficiently.

The CrewManager acts as the central controller, managing and coordinating both the ResearchAgent and WriterAgent.

The ResearchAgent is responsible for gathering and analyzing data, and it uses the DuckDuckGoSearchTool to fetch external information.

The WriterAgent generates structured outputs and reports using the ReportGenerator.

Both agents rely on the LLMConnector, which acts as a bridge to interact with large language models for intelligent processing.

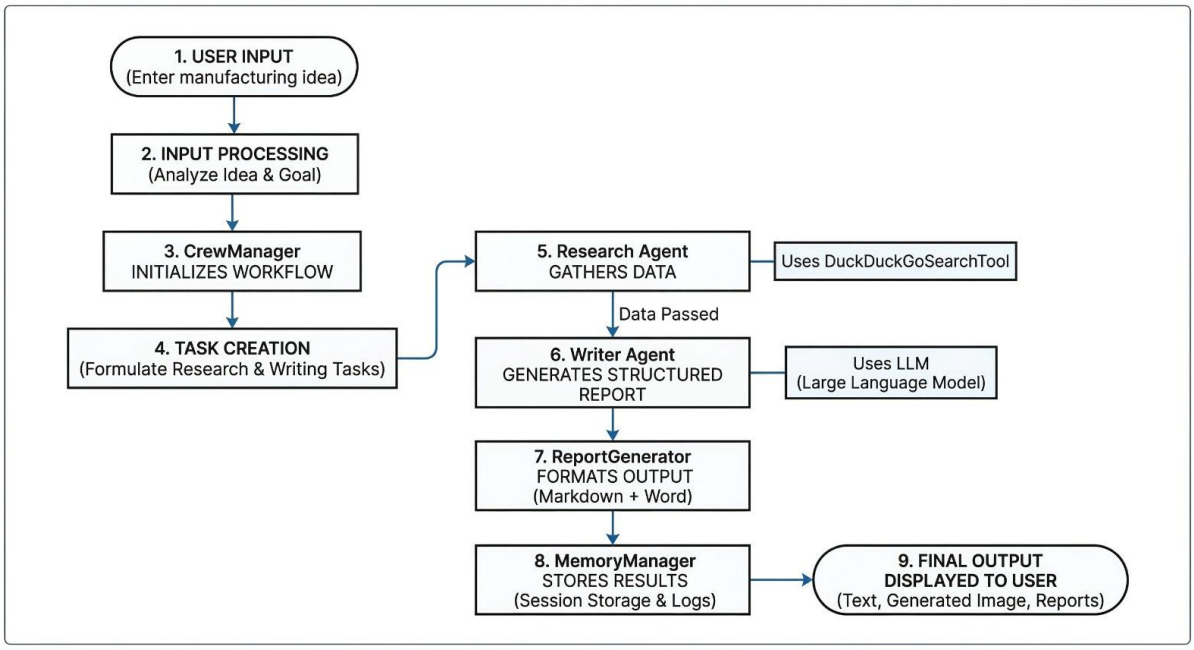
The MemoryManager stores outputs and intermediate data for future use, enabling context awareness.

The ConfigManager handles system configurations and settings.

The Task class (conceptually) represents the individual units of work assigned to agents.

Overall, the system follows a modular and scalable design, where agents perform specialized roles while being centrally coordinated, enabling efficient and intelligent task execution.

5.2 User Flow of the Application



Component & Process Breakdown

1. User Input (Entry Point)

Description: The starting point of the system where the user provides a natural language description of a manufacturing idea or project.

Function: Acts as the primary data source for the system's execution.

2. Input Processing (Analysis)

Description: A pre-processing layer that analyzes the user's raw text.

Function: It identifies specific goals, constraints, and the industry context, ensuring the data is clean and structured before being sent to the manager.

3. CrewManager (Workflow Initialization)

Description: The central orchestrator of the multi-agent environment.

Function: It receives the processed goal and "wakes up" the necessary agents. It sets the scope of the project and manages the transition between different execution phases.

4. Task Creation (Formulation)

Description: The logic layer where the global goal is broken down into atomic, executable tasks.

Function: It creates a "Research Task" and a "Writing Task" with specific instructions, ensuring each agent knows exactly what to produce.

5. Research Agent (Data Gathering)

Description: An autonomous agent specialized in information retrieval.

Function: It utilizes the DuckDuckGoSearchTool to fetch real-world data, market trends, or technical specifications from the web to ensure the final output is grounded in current facts.

6. Writer Agent (Report Generation)

Description: An autonomous agent specialized in content synthesis and professional writing.

Function: It receives the "Data Passed" from the Research Agent and uses a Large Language Model (LLM) to transform raw facts into a structured, professional manufacturing report.

7. ReportGenerator (Formatting Layer)

Description: A utility component designed for document styling.

Function: It takes the text output from the Writer Agent and converts it into multiple professional formats—specifically Markdown (for web/code viewing) and Word (for formal documentation).

8. MemoryManager (Data Persistence)

Description: The storage and logging interface of the system.

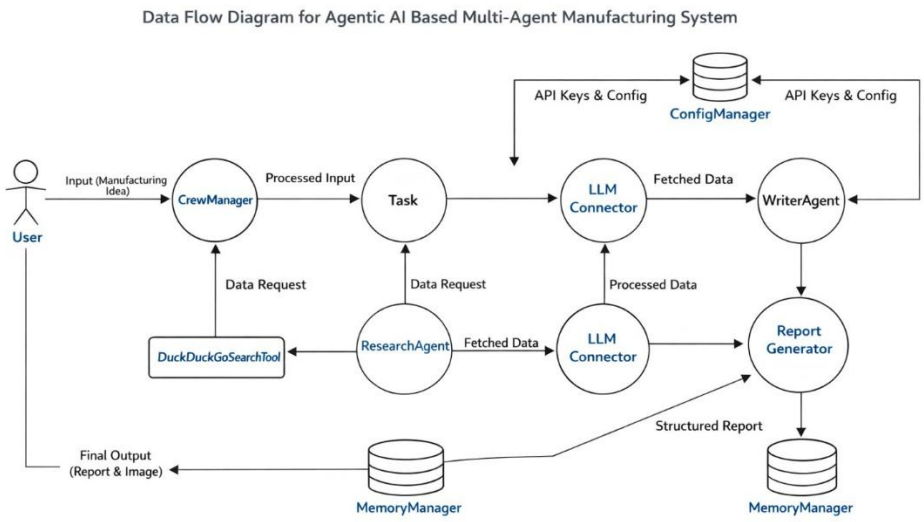
Function: It handles Session Storage and saves execution logs. This ensures that the system remembers what was done in previous steps and allows for data retrieval in future sessions.

9. Final Output (User Presentation)

Description: The termination point where the results are rendered to the user interface.

Function: It displays the processed Text, any Generated Images (visual schematics), and provides download links for the Reports, completing the user's request.

5.3 Data Flow Across Components (DFD)



The DFD shows the flow of data in the system: the user input is processed by CrewManager, tasks are handled by **ResearchAgent**, data is fetched via **DuckDuckGoSearchTool**, processed by the **LLM Connector**, structured by **WriterAgent**, and formatted by **ReportGenerator**. Results are stored in MemoryManager.

Data Flow Diagram

1. User (External Entity)

- The system starts with the **User ,** who provides input in the form of a **manufacturing idea**.
- This is the primary data source for the entire workflow.

2. CrewManager (Process)

- Receives the user input.
- Performs **initial processing and validation.**
- Controls the workflow and decides how tasks will be executed.
- Sends **processed input** to the Task module.

3. Task (Process)

- Represents a unit of work created from user input.
- Breaks down the idea into actionable steps.

- Sends data requests to the ResearchAgent.

4. ResearchAgent (Process)

- Responsible for collecting relevant information.
- Requests data using the **DuckDuckGoSearchTool**.
- Sends **fetched data** for further processing.

5. DuckDuckGoSearchTool (External Tool)

- Fetches real-time data from external sources (internet).
- Returns relevant information back to the ResearchAgent.

6. LLMConnector (Process)

- Acts as a bridge to AI/LLM models.
- Processes raw data into meaningful insights.
- Used at multiple stages:

1. After research (to process fetched data)
2. Before writing (to refine structured content)

7. WriterAgent (Process)

- Receives processed data from LLMConnector.
- Converts it into a structured report format.
- Focuses on clarity, organization, and readability.

8. ReportGenerator (Process)

- Takes structured content from WriterAgent.
- Formats it into:

1. Markdown
2. DOCX (Word format)
3. Prepares final presentation-ready output.

9. MemoryManager (Data Store)

- Stores results in structured formats (e.g., JSON).
- Saves:

1. Generated reports
2. Intermediate outputs
3. Helps in future retrieval and context reuse.

10. ConfigManager (Data Store / Support Component)

- Provides:

1. API keys
2. Configuration settings

3. Ensures secure and smooth communication between components (especially LLMConnector).

11. Data Flow Arrows (Important Understanding)

Each arrow represents movement of data:

Input→ User to CrewManager

Processed Input → CrewManager to Task

Data Request → Task to ResearchAgent

Fetched Data → ResearchAgent to LLMConnector

Processed Data → LLMConnector to WriterAgent

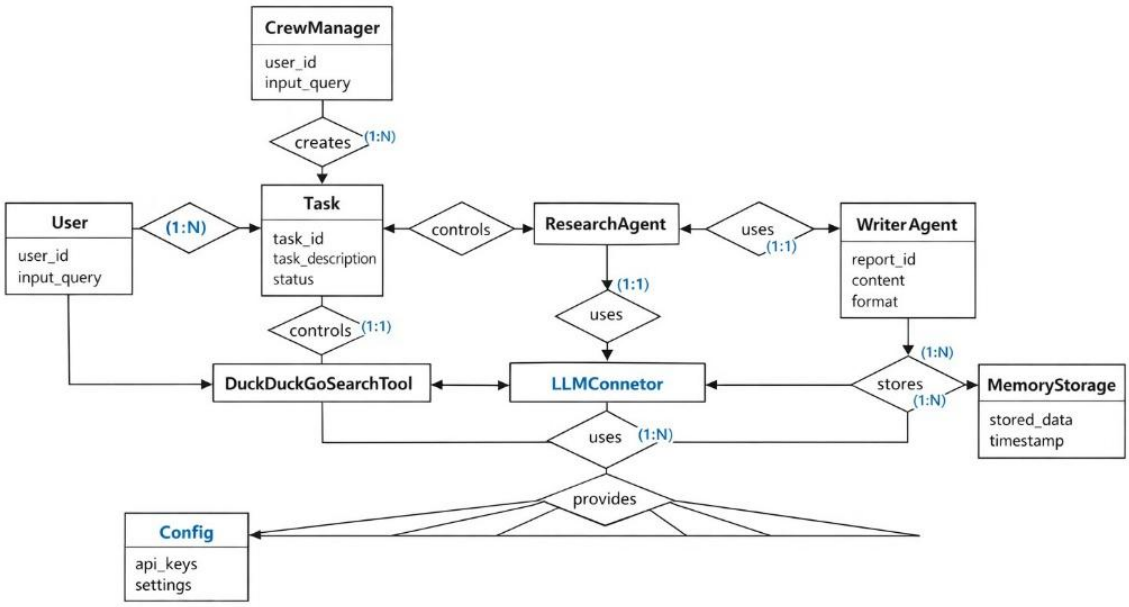
Structured Report → WriterAgent to ReportGenerator

Formatted Output → ReportGenerator to MemoryManager

Final Output → System to User

The system processes user input through coordinated agents, where data is researched, refined using LLMs, structured into reports, stored, and finally delivered as formatted output.

5.4 Entities and Relationships (ER Diagram)



ER Diagram

1. User (Entity)

- Attributes: user_id, input_query
- Represents the person interacting with the system.
- Relationship:

A User creates Tasks → (1:N)

→ One user can create multiple tasks.

2. Task (Entity)

- Attributes: task_id, task_description, status
- Represents a unit of work derived from user input.
- Relationships:

1. Created by User (1:N)
2. Managed by CrewManager
3. Connected to system processing flow

3. CrewManager (Entity)

- Acts as the central controller of the system.
- Relationships:

1. Manages Tasks
2. Controls ResearchAgent and WriterAgent
3. Coordinates execution of the workflow

4. ResearchAgent (Entity)

- Responsible for gathering and analyzing data.
- Relationships:

1. Controlled by CrewManager
2. Uses DuckDuckGoSearchTool (1:1) for data fetching
3. Uses LLMConnector (1:1) for processing data

5. WriterAgent (Entity)

- Responsible for generating structured reports.
- Relationships:

1. Controlled by CrewManager
2. Uses LLMConnector (1:1) to generate content
3. Generates Reports (1:N) → One agent can generate multiple reports

6. DuckDuckGoSearchTool (Entity)

- External tool used for retrieving real-time data.
- Relationship:

1. Used by ResearchAgent
2. Helps fetch relevant information from the web

7. LLMConnector (Entity)

- Connects the system to Large Language Models (LLMs).
- Relationships:

1. Used by both ResearchAgent and WriterAgent
2. Processes and refines data before report generation

8. Report (Entity)

- Attributes: report_id, content, format
- Represents the final structured output.
- Relationships:

1. Generated by WriterAgent
2. Stored in MemoryStorage

9. MemoryStorage (Entity / Data Store)

- Attributes: stored_data, timestamp
- Stores reports and processed results.
- Relationship:

1. Stores Reports (1:N) → One storage can hold many reports

10. Config (Entity)

- Attributes: api_keys, settings
- Holds configuration data required for system operation.
- Relationship:

Provides configuration to system components (especially LLMConnector and tools)

Relationship Summary (Quick View)

User → Task = creates (1:N)

CrewManager → Task = manages

CrewManager → Agents = controls

ResearchAgent → DuckDuckGoSearchTool = uses

Agents → LLMConnector = uses

WriterAgent → Report = generates

Report → MemoryStorage = stored

Config → System = provides settings

5.5 Technology Stack and Justification

This section outlines the technical architecture of the Agentic Multi-Agent Manufacturing System The stack is selected to prioritize high-speed inference, modular agent orchestration, and seamless data portability.

1. Core Language & Frameworks

Python (v3.10+): The industry standard for AI and data science, offering extensive libraries and a flexible environment for rapid prototyping.

CrewAI: A powerful framework for orchestrating "Role-Playing" autonomous agents, allowing for complex task delegation and collaborative workflows between the Research and Writer agents.

2. Intelligence & LLM Integration

Llama 3.3 (via Groq API): Utilized as the primary "brain" of the agents; it offers state-of-the-art reasoning capabilities with exceptionally low latency via Groq's LPU (Language Processing Unit) acceleration.

LiteLLM: Acts as a universal adapter to simplify API calls, ensuring the system remains provider-agnostic and allowing for easy swapping of LLM backends if needed.

3. Agentic Tools & Capabilities

DuckDuckGo Search Tool: Provides the Research Agent with real-time web access to gather the latest manufacturing trends and technical data without requiring complex API keys.

4. Data Storage & Persistence

JSON (Persistent Memory): A lightweight, human-readable format used to store agent states and memory, ensuring data can be easily retrieved and audited without the overhead of a full database.

5. Output & Delivery

Markdown: Used for structured, intermediate report generation and easy rendering in web-based environments or documentation.

Python-docx (Word/DOCX): A specialized library to programmatically generate professional, formatted reports suitable for stakeholders in the manufacturing sector.

6. Development Environment

VS Code: A versatile Integrated Development Environment (IDE) with robust support for Python debugging and extension-rich workflows.

GitHub: Used for version control and collaborative development, ensuring a clean history of the system's evolution.

5.6 Justification Summary Table

Category	Technology	Justification
Programming Language	Python 3.10+	Offers the best ecosystem for AI/ML and seamless integration with agentic frameworks.
Agent Orchestration	CrewAI	Allows for role-based task delegation and collaborative workflows between Research and Writer agents.
Inference Engine	Llama 3.3 (via Groq)	Provides high-speed reasoning and low-latency responses, which is critical for agentic loops.
Integration Layer	LiteLLM	Simplifies API management by providing a unified interface, making the system provider-agnostic.
Research Tool	DuckDuckGo Search	Enables agents to fetch real-time manufacturing data and news without needing complex API keys.

Data Persistence	JSON	A lightweight and human-readable format ideal for storing agent memory and state transitions.
Document Generation	Markdown & DOCX	Ensures the system can output both technical logs and professional-grade business reports.
Development Tools	VS Code & GitHub	Provides a robust environment for debugging, code management, and version control.

6. Conclusion

The Low Level Design (LLD) of the Multi-Agent Manufacturing System provides a detailed blueprint of the system's internal structure, including its classes, workflows, data flow, and component interactions. The design follows an Agentic AI architecture, where specialized agents collaborate efficiently to perform tasks such as research and report generation. The use of a well-defined technology stack, modular components, and structured data handling ensures that the system is scalable, maintainable, and efficient. The integration of multimodal capabilities further enhances the system's ability to generate both textual and visual outputs.

Overall, this LLD ensures a clear understanding of the system implementation and serves as a strong foundation for development, testing, and future enhancements.

7. Future Scope & Enhancements

The proposed system can be further enhanced to improve functionality, scalability, and real-world applicability.

Future Scope:

- Integration of advanced multimodal models for improved image and 3D prototype generation
- Expansion to support real-time manufacturing simulations and CAD-based designs
- Deployment on cloud platforms for scalability and multi-user access
- Addition of domain-specific agents (e.g., cost analysis, quality control, supply chain optimization)
- Integration with industrial databases and APIs for more accurate and real-time data

Enhancements:

- Implementation of user authentication and role-based access control
- Improvement in memory systems using vector databases for better semantic retrieval
- Enhancement of UI/UX for better user interaction and visualization
- Optimization of agent collaboration and task execution speed
- Support for multiple languages for broader accessibility